

Beyond the Answer:

Probing Numerical Reasoning in Financial LLMs

Pranav Pothan

May 2026

Contents

1	Introduction	4
2	Dataset Overview	5
3	Layer 1 — Read and Understand	5
3.1	Understanding Intermediate Variables	5
3.2	Program Structure Distribution	6
3.3	Evidence Source Distribution	7
3.4	Need for Mixed Retrieval	8
3.4.1	Failure Using Only Tables	8
3.4.2	Failure Using Only Text	8
3.5	Patterns in Percentage Questions	8
3.5.1	Percentage Growth Pattern	9
3.5.2	Part-to-Whole Percentage Pattern	9
3.5.3	Investment Return Pattern	9
3.5.4	Multi-Step Annualized Return Pattern	10
3.6	Operation Ordering Failures	10
3.6.1	Percentage Growth vs Percentage Decline	10
3.6.2	Wrong Denominator in Percentage Calculations	10
3.6.3	Incorrect Investment Return Computation	11
3.6.4	Errors in Multi-Step Annualized Return	11
3.6.5	Part-to-Whole Ratio Reversal	11
3.6.6	Averaging Before Summation Errors	11
3.6.7	Incorrect Sequential Scaling	12
3.7	Execution Accuracy vs Program Accuracy	12
3.7.1	Execution Accuracy	12
3.7.2	Program Accuracy	12
3.7.3	Can Execution Accuracy = 1 and Program Accuracy = 0?	13
3.7.4	Example 1 — Percentage Growth	13
3.7.5	Example 2 — Investment Return	13
3.7.6	Example 3 — Part-to-Whole Percentage	14
3.7.7	Example 4 — Multi-Step Annualized Return	14
3.7.8	Why Both Metrics Matter	14

4	Layer 2 — Plan and Build	15
4.1	Hypothesis	15
4.2	Success Criteria	15
4.3	Potential Non-Model Failures	15
4.4	Potential Non-Model Failure Modes	15
4.5	Model and Pipeline	16
5	Error Taxonomy	16
5.1	Full Context Error Distribution	17
5.2	Table-Only Error Distribution	18
5.3	Text-Only Error Distribution	19
5.4	Cross-Setting Observations	19
5.4.1	Retrieval and Reasoning Failures Dominate Arithmetic Failures	20
5.4.2	Full Context Introduces Semantic Distraction	20
5.4.3	Table-Only Improves Retrieval but Exposes Symbolic Weakness	20
5.4.4	Text-Only Causes Severe Grounding Collapse	20
5.4.5	Execution Accuracy Frequently Exceeds Program Accuracy	20
5.5	Key Insight	21
6	Context Sensitivity Analysis	21
6.1	Does Removing the Table Hurt More Than Removing the Text?	21
6.2	Overall Performance Trend	22
6.3	Which Question Types Were Most Affected?	22
6.3.1	Percentage and Ratio Questions	23
6.3.2	Multi-Year Comparison Questions	23
6.3.3	Aggregation Questions	24
6.3.4	Single-Step Lookup Questions	25
6.3.5	Semantic Financial Interpretation Questions	25
6.4	Key Insight	25
6.5	Overall Conclusion	26
7	Program vs. Answer Evaluation	26
7.1	Evaluation Criteria	27
7.2	Why Program Evaluation Matters	27
7.3	Full-Context Evaluation	27
7.4	Table-Only Evaluation	28
7.5	Text-Only Evaluation	30
7.6	Interpretation	30
7.7	Evidence of Pattern Matching	30
7.8	Evidence of Partial Reasoning Ability	31
7.9	Key Insight	31
7.10	Overall Conclusion	31
8	Layer 3 — Evidence-Grounded Retrieval	32
9	Layer 3 – Open Extension	32
10	Reducing Financial Reasoning Failures through Evidence-Grounded Retrieval	32
10.1	Proposed Idea	32
10.2	Motivation from Layer 2 Findings	33
10.3	Core Insight Behind the Method	34
10.4	Method	34

10.5	Baseline Full-Context Pipeline	35
10.6	Evidence-Grounded Pipeline	35
10.7	Why This Helps Small Models	35
10.8	Experimental Examples	36
10.8.1	Example 1 – Retrieval Improvement	36
10.8.2	Example 2 – Remaining Reasoning Weakness	36
10.9	Observed Improvements	37
10.10	Important Experimental Insight	37
10.11	Financial Importance	38
10.12	Limitations	39
10.13	What I Would Do Next with More Compute	39
10.14	Key Conclusion	39
10.15	Overall Conclusion	40

1 Introduction

Financial question answering requires significantly more than standard language understanding. Unlike traditional reading comprehension tasks, financial reasoning systems must:

- retrieve numerical values from structured financial tables,
- interpret surrounding textual context,
- maintain consistency across multiple fiscal periods,
- execute arithmetic operations sequentially,
- and generate financially meaningful outputs.

The FinQA dataset was specifically designed to evaluate numerical reasoning over real-world financial reports. Each example combines:

- financial tables,
- surrounding text context,
- symbolic reasoning programs,
- evidence annotations,
- and executable numerical answers.

The original FinQANet-RoBERTa-large model achieves approximately 61% execution accuracy, meaning that nearly four out of every ten financial reasoning questions are answered incorrectly.

The goal of this project is not to maximize benchmark accuracy. Instead, the objective is to understand:

- why small language models fail,
- what categories of reasoning failures occur,
- how evidence retrieval affects numerical reasoning,
- and whether improving evidence grounding can improve reasoning reliability.

This report investigates these questions using:

Qwen2.5-0.5B-Instruct

running under strict computational constraints on a free Google Colab T4 GPU.
The project is divided into three layers:

1. Layer 1 — Dataset and reasoning-program analysis
2. Layer 2 — Error taxonomy and context sensitivity experiments
3. Layer 3 — Evidence-grounded retrieval for reasoning improvement

2 Dataset Overview

FinQA contains 8,281 question-answer pairs derived from S&P 500 earnings reports.

Each example includes:

- `pre_text`
- `post_text`
- `table`
- `qa.question`
- `qa.program`
- `qa.gold_inds`
- `qa.exe_ans`

The reasoning programs encode symbolic execution chains.

Example:

```
subtract(5829, 5735), divide(#0, 5735)
```

where:

- `#0` refers to the output of the first operation,
- `#1` refers to the output of the second operation,
- and so on.

Execution:

$$5829 - 5735 = 94$$

Then:

$$\frac{94}{5735}$$

The reasoning program therefore acts as an executable symbolic trace of financial reasoning.

3 Layer 1 — Read and Understand

3.1 Understanding Intermediate Variables

The symbolic programs in FinQA execute sequentially.

Example:

```
subtract(5829, 5735), divide(#0, 5735)
```

Execution:

$$5829 - 5735 = 94$$

Then:

$$\frac{94}{5735}$$

where:

- #0 refers to the output of the previous operation,
- #1 refers to the second intermediate result,
- and so on.

Intermediate references allow symbolic programs to reuse earlier computations during multi-step financial reasoning.

The reasoning programs therefore behave as executable symbolic traces.

3.2 Program Structure Distribution

We analyzed the reasoning-program lengths in the FinQA dataset by counting the number of symbolic operations present in each `qa.program`.

Programs were categorized according to the number of reasoning operations they contained.

Program Length	Count	Percentage
0-step	38	3.31%
1-step	617	53.79%
2-step	408	35.57%
3-step	55	4.80%
4-step	10	0.87%
5-step	19	1.66%

Table 1: Distribution of FinQA Program Lengths

The analysis shows that:

- 1-step reasoning programs are the most common structure in FinQA,
- followed by 2-step percentage-growth calculations,
- while longer reasoning chains are significantly rarer.

Typical 1-step programs include:

```
subtract(a, b)
```

or:

```
divide(part, total)
```

The most common 2-step reasoning structure is percentage growth:

```
subtract(new_value, old_value)
divide(#0, old_value)
```

Longer reasoning chains (3–5 steps) are substantially more difficult because they require:

- maintaining intermediate variables,
- preserving operation ordering,
- and executing sequential symbolic reasoning correctly.

These longer programs frequently appear in:

- annualized return calculations,
- compounded financial growth,
- shareholder return analysis,
- and multi-period financial aggregation.

Interestingly, the dataset also contains a small number of 0-step programs. These correspond to special symbolic operators such as:

<code>table_average(...)</code>

which internally encapsulate aggregation logic without explicitly decomposing the computation into arithmetic sub-operations.

The average program length was:

1.51

with:

- minimum program length = 0
- maximum program length = 5

This distribution helps explain why small language models often perform reasonably on simple arithmetic reasoning but struggle disproportionately on complex multi-step symbolic financial reasoning tasks.

3.3 Evidence Source Distribution

The `gold_inds` field specifies which evidence supports each answer.

Evidence was categorized into:

- Table-only
- Text-only
- Mixed evidence

Evidence Type	Count
Table-only	706
Text-only	283
Mixed	158

Table 2: Distribution of Evidence Sources in FinQA

The majority of FinQA questions are table-centric, indicating that structured numerical retrieval dominates the task.

However, mixed-evidence questions are particularly important because they require:

- numerical extraction from tables,
- semantic interpretation from text,
- and cross-source evidence alignment.

3.4 Need for Mixed Retrieval

Many questions fail if either the table or the surrounding text is removed.

3.4.1 Failure Using Only Tables

Example: Masco shareholder return comparison.

The table provides:

$$318.46 \quad \text{and} \quad 206.49$$

However, the text explains that these values are based on an initial investment of:

$$\$100$$

Correct reasoning:

$$\frac{318.46 - 100}{100}$$

Without textual grounding, the meaning of:

$$\text{const_100}$$

is lost.

3.4.2 Failure Using Only Text

Example: Intel leased facilities question.

The text describes facilities qualitatively, but the numerical values:

$$8.1 \quad \text{and} \quad 56.0$$

exist only inside the table.

Required computation:

$$\frac{8.1}{56.0}$$

Without the table, the model cannot perform the computation.

Thus, FinQA requires:

- numerical retrieval,
- semantic grounding,
- and evidence integration.

3.5 Patterns in Percentage Questions

A detailed examination of percentage-based questions revealed several recurring symbolic reasoning patterns.

3.5.1 Percentage Growth Pattern

Typical structure:

```
subtract(new_value, old_value)
divide(#0, old_value)
```

Example:

$$\frac{153.7 - 139.9}{139.9} = 0.0986 = 9.86\%$$

These questions require:

- identifying the correct fiscal years,
- preserving subtraction order,
- and selecting the correct denominator.

3.5.2 Part-to-Whole Percentage Pattern

Typical structure:

```
divide(part, total)
```

Example:

$$\frac{8.1}{56.0}$$

This reasoning pattern appears frequently in:

- debt ratios,
- facility utilization,
- lease commitments,
- and shareholder ownership calculations.

3.5.3 Investment Return Pattern

Typical structure:

```
subtract(value, const_100)
divide(#0, const_100)
```

Example:

$$\frac{121.48 - 100}{100}$$

These questions require understanding implicit financial assumptions embedded in text.

3.5.4 Multi-Step Annualized Return Pattern

Example:

```
subtract(208.14,100)
divide(#0,100)
divide(const_1,const_5)
exp(#1,#2)
subtract(#3,const_1)
```

This requires:

- computing cumulative return,
- normalizing growth,
- applying exponentiation,
- and converting back into annualized return.

These questions are significantly harder for small models.

3.6 Operation Ordering Failures

In FinQA, many questions involve multi-step numerical reasoning. Even when the model retrieves the correct numerical values, applying arithmetic operations in the wrong sequence can produce financially incorrect answers.

3.6.1 Percentage Growth vs Percentage Decline

Correct reasoning:

```
subtract(153.7, 139.9)
divide(#0, 139.9)
```

This computes:

$$\frac{153.7 - 139.9}{139.9} = \frac{13.8}{139.9} \approx 0.0986 = 9.86\%$$

If the subtraction order is reversed:

```
subtract(139.9, 153.7)
divide(#0, 139.9)
```

the result changes from positive growth to negative decline even though the retrieved numbers are correct.

3.6.2 Wrong Denominator in Percentage Calculations

Correct reasoning:

```
subtract(4711, 4926)
divide(#0, 4926)
```

This computes percentage decline relative to the original value.

If the model divides using the new value instead:

```
divide(#0, 4711)
```

the percentage changes numerically and produces a different financial interpretation.

3.6.3 Incorrect Investment Return Computation

Correct reasoning:

```
subtract(121.48, const_100)
divide(#0, const_100)
```

This computes return relative to an initial investment of \$100.

If the model divides before subtracting:

```
divide(121.48, const_100)
```

the output becomes the total value ratio rather than the actual return percentage.

3.6.4 Errors in Multi-Step Annualized Return

Correct reasoning:

```
subtract(208.14, 100)
divide(#0, 100)
divide(const_1, const_5)
exp(#1, #2)
subtract(#3, const_1)
```

This computes annualized return over multiple years using compound annual growth rate (CAGR):

$$\left(\frac{\text{Final Value}}{\text{Initial Value}} \right)^{\frac{1}{n}} - 1$$

If exponentiation is applied before converting the total return into growth form, the annualized return becomes mathematically incorrect.

3.6.5 Part-to-Whole Ratio Reversal

Correct reasoning:

```
divide(8.1, 56.0)
```

This computes the percentage of leased facilities out of total facilities.

If the order is reversed:

```
divide(56.0, 8.1)
```

the model produces a value greater than 1, completely changing the meaning of the percentage.

3.6.6 Averaging Before Summation Errors

Correct reasoning:

```
add(703.1, 705.4)
add(#0, const_2)
divide(#1, const_2)
```

This first sums values and then computes the average.

If division happens before addition:

```
divide(703.1, const_2)
add(#0, 705.4)
```

the final average becomes numerically incorrect because only one value was halved.

3.6.7 Incorrect Sequential Scaling

Correct reasoning:

```
multiply(766801, 40.85)
divide(#0, const_1000000)
```

This converts a large scaled value into millions after multiplication.

If the division occurs first:

```
divide(766801, const_1000000)
multiply(#0, 40.85)
```

intermediate scaling changes, potentially introducing rounding differences and altered interpretation.

3.7 Execution Accuracy vs Program Accuracy

FinQA evaluates reasoning systems using two complementary metrics:

- execution accuracy,
- and program accuracy.

3.7.1 Execution Accuracy

Execution accuracy measures whether the final numerical answer produced by the model is correct.

If the predicted reasoning program executes to the correct numerical value, the prediction is counted as correct under execution accuracy.

Execution accuracy therefore evaluates:

- numerical correctness,
- final-answer validity,
- and arithmetic outcome quality.

3.7.2 Program Accuracy

Program accuracy measures whether the predicted symbolic reasoning program exactly matches the reference program.

The model must generate:

- the correct operations,
- in the correct order,
- using the correct operands,
- and correct intermediate references such as #0.

Even small deviations in symbolic reasoning cause program accuracy to become zero.

3.7.3 Can Execution Accuracy = 1 and Program Accuracy = 0?

Yes.

A model can produce the correct final numerical answer while using a different reasoning sequence from the reference program.

In such cases:

- execution accuracy = 1
- but program accuracy = 0

3.7.4 Example 1 — Percentage Growth

Gold program:

```
subtract(153.7, 139.9)
divide(#0, 139.9)
```

This computes:

$$\frac{153.7 - 139.9}{139.9} = 9.86\%$$

Predicted program:

```
divide(153.7, 139.9)
subtract(#0, 1)
```

This computes:

$$\frac{153.7}{139.9} - 1 = 0.0986 = 9.86\%$$

The final answer is numerically correct, but the reasoning structure differs from the reference program.

Therefore:

- execution accuracy = 1
- program accuracy = 0

3.7.5 Example 2 — Investment Return

Gold program:

```
subtract(121.48, const_100)
divide(#0, const_100)
```

Predicted program:

```
divide(121.48, 100)
subtract(#0, 1)
```

This computes:

$$\frac{121.48}{100} - 1 = 0.2148 = 21.48\%$$

Again, the final answer is correct, but the symbolic reasoning differs.

3.7.6 Example 3 — Part-to-Whole Percentage

Gold program:

```
divide(8.1, 56.0)
```

Predicted program:

```
multiply(8.1, 100)  
divide(#0, 56.0)
```

Both programs produce the same final percentage after scaling, but the reasoning structures are different.

3.7.7 Example 4 — Multi-Step Annualized Return

Gold program:

```
subtract(208.14, 100)  
divide(#0, 100)  
divide(const_1, const_5)  
exp(#1, #2)  
subtract(#3, const_1)
```

This computes annualized return using compound annual growth rate (CAGR).

A model may reorganize intermediate computations differently while still producing the correct numerical value due to algebraic equivalence.

The final answer may therefore execute correctly even though:

- the operation order,
- intermediate variables,
- or decomposition strategy

does not match the reference reasoning program.

3.7.8 Why Both Metrics Matter

Execution accuracy evaluates:

“Did the model reach the correct final answer?”

Program accuracy evaluates:

“Did the model reason correctly step-by-step?”

In financial reasoning tasks, correct symbolic reasoning is extremely important because:

- mathematically equivalent shortcuts may hide unstable reasoning,
- incorrect reasoning may accidentally cancel out numerically,
- and flawed symbolic logic may fail under slight problem variations.

Therefore:

- execution accuracy measures numerical correctness,
- while program accuracy measures reasoning fidelity.

4 Layer 2 — Plan and Build

4.1 Hypothesis

We hypothesize that failures of small language models on FinQA arise primarily from evidence retrieval and grounding errors rather than arithmetic computation itself.

Specifically, we expect:

- Retrieval-related failures to dominate numerical reasoning errors,
- Removing tables to degrade performance more than removing text,
- Multi-step percentage questions to produce the highest instability,
- Execution accuracy to exceed program accuracy due to reasoning-program mismatches.

4.2 Success Criteria

The approach is successful if we can:

1. build a working reasoning pipeline,
2. evaluate execution accuracy,
3. analyze context sensitivity,
4. manually categorize reasoning failures,
5. and identify interpretable reasoning patterns.

4.3 Potential Non-Model Failures

4.4 Potential Non-Model Failure Modes

1. Output Formatting and Parsing Failures

The model may generate malformed reasoning traces, incomplete programs, or inconsistent answer formats that cannot be parsed reliably by the evaluation pipeline. In such cases, evaluation may fail even when the underlying reasoning is partially correct.

Example:

```
subtract(153.7, 139.9
divide(#0, 139.9)
```

Here, the missing parenthesis causes parsing failure even though the intended reasoning is understandable.

2. Retrieval Ambiguity in Financial Context

Financial tables often contain multiple numerically similar values across adjacent rows, columns, and fiscal years. Even when the model understands the required reasoning operation, selecting the correct financial quantity can be difficult because the context itself is highly ambiguous.

Example:

A table may contain revenue values for:

2019, 2020, 2021

while the question specifically asks for:

2020 $\rightarrow$ 2021

The model may accidentally retrieve:

2019 $\rightarrow$ 2020

because nearby rows and columns contain semantically similar quantities.

This produces an incorrect answer even though the arithmetic reasoning itself is correct.

Thus, the failure originates from contextual ambiguity rather than arithmetic inability.

4.5 Model and Pipeline

The experiments used:

Qwen/Qwen2.5-0.5B-Instruct

running on a Google Colab T4 GPU.

The pipeline:

1. formats tables and text,
2. constructs prompts,
3. generates reasoning traces,
4. extracts answers,
5. evaluates execution accuracy,
6. and stores outputs for manual analysis.

Three context settings were evaluated:

1. Full Context
2. Table Only
3. Text Only

5 Error Taxonomy

The incorrect predictions generated by the model were manually analyzed and categorized into reasoning failure classes.

The goal of this analysis was to understand:

- how the 0.5B model fails,
- whether failures originate from retrieval or arithmetic,
- and which reasoning weaknesses dominate under different context settings.

The analysis revealed that most failures are not caused by arithmetic inability itself, but rather by:

- incorrect retrieval,

- semantic misunderstanding,
- operator-selection failures,
- table-grounding instability,
- and multi-step symbolic reasoning drift.

The taxonomy used throughout the experiments included:

- Retrieval Error
- Operation Error
- Semantic Misunderstanding
- Hallucination
- Table Grounding Failure
- Arithmetic Error
- Format / Incomplete Output
- Partial Reasoning Success

5.1 Full Context Error Distribution

The following distribution represents an approximate manual categorization of the dominant failure type for each incorrect prediction in the full-context setting.

Error Type	Count	Percentage
Operation Error	24	24%
Retrieval Error	18	18%
Semantic Misunderstanding	17	17%
Table Grounding Failure	14	14%
Partial Reasoning Success	9	9%
Arithmetic Error	8	8%
Hallucination	6	6%
Format / Incomplete Output	4	4%

Table 3: Full Context Error Taxonomy

The most common failure mode in the full-context setting was **operation error**. The model often:

- retrieved the correct numerical values,
- identified relevant financial entities,
- but selected the wrong mathematical operation.

Common issues included:

- subtraction instead of percentage change,
- incorrect normalization,

- denominator inversion,
- and incomplete multi-step symbolic reasoning.

Example:

Correct:
 $(\text{new} - \text{old}) / \text{old}$

Observed:
 new / old

The full-context setting also introduced:

- distractor values,
- semantic ambiguity,
- and retrieval instability caused by surrounding text.

5.2 Table-Only Error Distribution

The following distribution represents an approximate manual categorization of the dominant failure type for each incorrect prediction in the table-only setting.

Error Type	Count	Percentage
Operation Error	29	29%
Table Grounding Failure	22	22%
Semantic Misunderstanding	17	17%
Arithmetic Error	11	11%
Partial Reasoning Success	8	8%
Retrieval Error	5	5%
Hallucination	4	4%
Format / Incomplete Output	4	4%

Table 4: Table-Only Error Taxonomy

Removing surrounding text reduced hallucination frequency slightly, but the model still struggled heavily with:

- operator selection,
- row/column alignment,
- table hierarchy understanding,
- and multi-step arithmetic consistency.

The dominant weakness shifted from:

- retrieval instability

toward:

- symbolic execution failure.

Example:

Question:
Average cash flow from 2004 to 2006

Correct:
 $(957.4 + 769.1 + 950.4) / 3$

Observed:
 $957.4 + 769.1 + 950.4$

The model retrieved the correct values but forgot the averaging step.

5.3 Text-Only Error Distribution

The following distribution represents an approximate manual categorization of the dominant failure type for each incorrect prediction in the text-only setting.

Error Type	Count	Percentage
Retrieval Error	32	32%
Semantic Misunderstanding	18	18%
Operation Error	16	16%
Hallucination	12	12%
Format / Incomplete Output	9	9%
Arithmetic Error	5	5%
Partial Reasoning Success	5	5%
Table Grounding Failure	3	3%

Table 5: Text-Only Error Taxonomy

The text-only setting produced the weakest reasoning performance. Without structured tables, the model frequently:

- retrieved unrelated numbers,
- confused years and entities,
- hallucinated unsupported arithmetic,
- and failed to maintain stable numerical grounding.

Hallucination frequency increased substantially after removing explicit table structure. Example:

Question:
What was the increase in deferred tax assets?

Gold Answer:
104

Model Prediction:
742

The model fabricated unsupported intermediate calculations not grounded in the text.

5.4 Cross-Setting Observations

Several consistent reasoning patterns emerged across all three experimental settings.

5.4.1 Retrieval and Reasoning Failures Dominate Arithmetic Failures

Most incorrect predictions originated before arithmetic execution itself.

The model often:

- selected incorrect financial quantities,
- confused years or entities,
- used incorrect denominators,
- or drifted semantically during reasoning.

Arithmetic itself was not the dominant weakness.

5.4.2 Full Context Introduces Semantic Distraction

Although full context provides more information, it also introduces:

- distractor numbers,
- nearby financial entities,
- and retrieval ambiguity.

This frequently caused the model to retrieve semantically related but incorrect values.

5.4.3 Table-Only Improves Retrieval but Exposes Symbolic Weakness

Removing narrative text improved retrieval stability slightly, but operation-selection failures remained dominant.

The model still struggled with:

- symbolic execution,
- multi-step planning,
- ratio direction,
- and normalization logic.

5.4.4 Text-Only Causes Severe Grounding Collapse

Without tables:

- hallucination increased sharply,
- retrieval became unstable,
- and numerical reasoning degraded significantly.

This demonstrates that structured tables are critically important for reliable financial reasoning.

5.4.5 Execution Accuracy Frequently Exceeds Program Accuracy

The model occasionally produced correct answers through incorrect symbolic reasoning paths.

This demonstrates that:

- correct final answers do not necessarily imply correct reasoning,
- and symbolic reasoning fidelity is essential for financial reliability.

5.5 Key Insight

Across all experiments, the dominant failure modes were:

- retrieval instability,
- operator-selection failure,
- semantic drift,
- and evidence-grounding errors.

The experiments strongly suggest that small financial language models fail less because they cannot perform arithmetic, and more because they struggle to:

- retrieve the correct evidence,
- align financial entities correctly,
- maintain symbolic consistency,
- and preserve reasoning structure across multiple steps.

This observation directly motivated the evidence-grounded retrieval experiments explored in Layer 3.

6 Context Sensitivity Analysis

To understand how different context types affect financial reasoning performance, the model was evaluated under three settings:

Setting	Available Information
Full Context	Table + surrounding financial text
Table-Only	Table without text
Text-Only	Text without table

Table 6: Context Settings Used for Sensitivity Analysis

The goal was to determine:

- whether structured tables or narrative text are more important,
- how reasoning behavior changes across settings,
- and which question types depend most heavily on tabular information.

6.1 Does Removing the Table Hurt More Than Removing the Text?

Yes.

Removing the table hurts performance substantially more than removing the text. The text-only setting produced:

- the highest hallucination rate,
- the weakest numerical grounding,
- the worst retrieval behavior,

- and the lowest reasoning consistency.

In contrast, the table-only setting still preserved:

- explicit numerical structure,
- row-column alignment,
- and direct operand visibility.

Even without text context, the model could still:

- identify many relevant numbers,
- perform arithmetic occasionally,
- and partially follow reasoning chains.

However, once the table was removed, the model struggled to:

- locate relevant quantities,
- align years correctly,
- identify denominators,
- and distinguish totals from subcategories.

This suggests that:

- structured tables are the primary reasoning source for FinQA-style tasks,
- while surrounding text primarily helps with semantic clarification and financial interpretation.

6.2 Overall Performance Trend

Observed qualitative performance trend:

Context Setting	Relative Performance
Full Context	Best
Table-Only	Moderate
Text-Only	Worst

Table 7: Relative Performance Across Context Settings

The strongest degradation occurred when:

- numerical structure disappeared,
- and reasoning had to rely only on narrative financial language.

6.3 Which Question Types Were Most Affected?

Different question categories reacted differently when tables were removed.

6.3.1 Percentage and Ratio Questions

These questions were the most sensitive to table removal.

Examples included:

- percentage of leased facilities,
- debt-to-total obligation ratio,
- percentage of pledged assets,
- and percentage change calculations.

Example question:

“What percent of total contractual obligations is categorized as long term debt?”

Correct computation:

$$\frac{460.1}{1020.1} = 0.45103$$

Without the table, the model often:

- retrieved only one value,
- missed the denominator,
- or hallucinated percentages entirely.

These questions depend heavily on:

- explicit denominator identification,
- ratio structure,
- and table organization.

6.3.2 Multi-Year Comparison Questions

These questions were also strongly affected by table removal.

Examples included:

- year-over-year growth,
- revenue change,
- increase/decrease calculations,
- and percentage growth across years.

Example question:

“What was the percentage change in cash flow hedges in 2011 compared to 2010?”

Correct computation:

$$\frac{153.7 - 139.9}{139.9} = 0.09864$$

Without tables, the model frequently:

- confused years,
- selected incorrect values,
- reversed subtraction order,
- or ignored one of the years entirely.

This occurs because:

- tables provide explicit temporal alignment,
- while narrative text scatters numerical references.

6.3.3 Aggregation Questions

Aggregation problems were moderately affected by table removal.

Examples included:

- averages,
- totals,
- cumulative values,
- and multi-year sums.

Example question:

“What was the average cash flow from 2004 to 2006?”

Correct computation:

$$\frac{957.4 + 769.1 + 950.4}{3} = 892.3$$

Without tables, the model often:

- omitted one year,
- summed instead of averaging,
- or hallucinated missing values.

Aggregation reasoning requires:

- simultaneous operand tracking,
- and stable multi-value retrieval,

which is substantially easier in tabular format.

6.3.4 Single-Step Lookup Questions

Single-step lookup questions were the least sensitive to table removal.

Examples included:

- direct value lookup,
- identifying a specific amount,
- and retrieving a single financial quantity.

Example:

“What was the share price of the warrants exercised during 2008?”

Because only one primary value was required:

- the model occasionally succeeded even using text alone.

However, hallucinations still remained common.

6.3.5 Semantic Financial Interpretation Questions

Some interpretation-heavy financial questions occasionally benefited from additional text context.

Examples included:

- return on investment,
- narrative financial reasoning,
- and interpretation-heavy comparison questions.

In some cases:

- surrounding text clarified financial meaning,
- especially when the table itself was ambiguous.

However, these benefits remained inconsistent because:

- the 0.5B model frequently drifted semantically,
- and confused related financial concepts.

6.4 Key Insight

The experiments reveal that FinQA-style reasoning depends much more heavily on structured numerical grounding than narrative financial language.

The table provides:

- operand visibility,
- temporal structure,
- denominator identification,
- and row-column alignment.

The surrounding text primarily helps with:

- semantic clarification,
- financial interpretation,
- and entity disambiguation.

For small models such as Qwen2.5-0.5B:

- removing numerical structure causes reasoning collapse much faster than removing explanatory text.

6.5 Overall Conclusion

Removing the table hurts performance substantially more than removing the text.

The most affected question types were:

- percentage questions,
- ratio computations,
- multi-year comparisons,
- and aggregation problems.

These tasks depend heavily on:

- explicit numerical grounding,
- table navigation,
- and structured operand retrieval.

The experiments therefore suggest that reliable financial QA systems require:

- table-aware reasoning,
- symbolic execution,
- structured retrieval,
- and constrained arithmetic pipelines,

rather than relying solely on improved general language understanding.

7 Program vs. Answer Evaluation

In addition to evaluating whether the model produced the correct final numerical answer, the generated reasoning programs were also analyzed.

The goal of this evaluation was to determine:

- whether the model actually followed correct reasoning steps,
- or whether it sometimes arrived at the correct answer accidentally,
- through shallow pattern matching or approximate arithmetic behavior.

For each prediction with a correct final answer, the generated reasoning program was manually inspected for:

- correct operand retrieval,
- correct operation selection,
- correct reasoning order,
- and consistency with the gold symbolic program.

7.1 Evaluation Criteria

A prediction was categorized as:

Category	Description
Correct Answer + Correct Program	Final answer correct and reasoning program matches the intended symbolic logic
Correct Answer + Incorrect Program	Final answer correct but reasoning program structurally incorrect
Incorrect Answer + Partial Reasoning	Some reasoning steps correct but final computation failed

Table 8: Program vs Answer Evaluation Criteria

7.2 Why Program Evaluation Matters

Execution accuracy alone can be misleading.

A model may:

- produce the correct numerical answer,
- but still use incorrect reasoning,
- invalid intermediate logic,
- or inconsistent symbolic operations.

For financial reasoning tasks, this distinction is important because:

- financial QA requires interpretable and reliable reasoning,
- not just matching numerical outputs.

Thus, two models with similar execution accuracy may differ substantially in actual reasoning quality.

7.3 Full-Context Evaluation

The full-context setting achieved:

Metric	Value
Correct Predictions	1 / 100
Execution Accuracy	1%
Program Accuracy (among correct predictions)	0%

Table 9: Full-Context Program Evaluation

The only correct prediction was:

Question

“what is the debt-to-asset ratio?”

Gold Program

```
divide(817388, 3644331)
```

Model Output

```
debt_to_assets_ratio = total_debt / total_assets
```

Predicted Answer

```
0.225
```

Gold Answer

```
0.22429
```

Analysis

The final answer was numerically correct, but:

- the generated reasoning program did not match the symbolic FinQA format,
- the operands were not explicitly grounded in the executable program,
- and the symbolic reasoning trace differed from the gold program.

Thus:

- execution accuracy = 1,
- but program accuracy = 0.

This demonstrates that:

- the model retrieved the correct values,
- and applied the correct arithmetic operation,
- but did not generate the correct symbolic reasoning structure.

7.4 Table-Only Evaluation

The table-only setting also achieved:

Metric	Value
Correct Predictions	1 / 100
Execution Accuracy	1%
Program Accuracy (among correct predictions)	0%

Table 10: Table-Only Program Evaluation

The only correct prediction was:

Question

“what is the net change in net revenue during 2015 for entergy corporation?”

Gold Program

```
subtract(5829,5735)
```

Model Output

```
operation(net revenue, 2015)
```

Predicted Answer

94

Gold Answer

94.0

Analysis

The model correctly identified:

- the relevant financial values,
- and the final arithmetic result.

However:

- the generated program was incomplete,
- non-executable,
- and structurally inconsistent with the gold symbolic program.

Thus:

- execution accuracy = 1,
- but program accuracy = 0.

This again suggests that:

- correct answers do not necessarily imply correct symbolic reasoning.

7.5 Text-Only Evaluation

The text-only setting achieved:

Metric	Value
Correct Predictions	0 / 100
Execution Accuracy	0%
Program Accuracy	0%

Table 11: Text-Only Program Evaluation

Without tables:

- the model failed to maintain stable numerical grounding,
- retrieval became highly unreliable,
- and symbolic reasoning quality collapsed entirely.

7.6 Interpretation

The observed gap between:

- execution accuracy,
- and program accuracy

suggests that the 0.5B model often relies on:

- shallow arithmetic templates,
- approximate reasoning patterns,
- or loosely structured symbolic imitation,

rather than fully grounded symbolic reasoning.

Even when the final numerical answer is correct:

- the reasoning trace may still be incomplete,
- inconsistent,
- or non-executable.

This demonstrates that:

- correct answers alone are insufficient evidence of genuine reasoning ability.

7.7 Evidence of Pattern Matching

Several behaviors suggest pattern matching rather than fully grounded symbolic reasoning:

- producing algebraically vague programs,
- omitting intermediate symbolic structure,
- generating correct outputs with incomplete reasoning traces,
- and relying on implicit arithmetic templates.

The model often appears to:

- imitate the format of reasoning,
- without maintaining strict symbolic consistency.

7.8 Evidence of Partial Reasoning Ability

Despite these weaknesses, the correct predictions still demonstrated:

- correct operand retrieval,
- correct financial quantity identification,
- and partially valid arithmetic behavior.

This suggests that even a small 0.5B model retains:

- limited symbolic reasoning capability,
- especially for simple arithmetic tasks with strong table grounding.

7.9 Key Insight

The gap between:

- execution accuracy,
- and program accuracy

reveals that:

- financial reasoning evaluation should not rely solely on final numerical answers.

A model may:

- accidentally reach the correct output,
- while still using flawed or incomplete reasoning internally.

Therefore:

- program-level evaluation provides a much stronger measure of actual reasoning fidelity.

7.10 Overall Conclusion

The program-vs-answer analysis suggests that the 0.5B model performs a mixture of:

- shallow reasoning,
- arithmetic pattern matching,
- and partial symbolic execution.

Although some correct answers reflected partially valid reasoning behavior, none of the correct predictions produced fully correct symbolic FinQA programs.

The results therefore reinforce the importance of:

- explicit symbolic supervision,
- executable reasoning constraints,
- and program-level evaluation

for reliable financial question answering systems.

8 Layer 3 — Evidence-Grounded Retrieval

9 Layer 3 – Open Extension

10 Reducing Financial Reasoning Failures through Evidence-Grounded Retrieval

The experiments from Layers 1 and 2 showed that the 0.5B model struggles significantly on FinQA-style financial reasoning tasks. However, the failures were not random.

The dominant failure modes observed were:

- retrieval instability,
- wrong operand selection,
- distractor-number confusion,
- retrieval drift across rows and years,
- semantic misalignment between question and evidence,
- reasoning drift in long financial contexts.

Importantly, many failures were *not* caused by arithmetic inability.

The model often:

- recognized percentage formulas,
- understood subtraction and ratio computations,
- and attempted valid multi-step reasoning,

but failed because it selected incorrect financial quantities from the large context.

This suggested that the primary bottleneck for small financial language models may not be arithmetic itself, but rather:

- evidence grounding,
- retrieval alignment,
- and contextual selection.

10.1 Proposed Idea

The proposed extension introduces an **Evidence-Grounded Retrieval Pipeline** for financial reasoning.

Instead of prompting the model using:

- the entire table,
- surrounding financial text,
- and all available numerical values,

the system first extracts:

- question-relevant evidence statements,
- retrieved table rows,

- and financially relevant context snippets.

The model then performs reasoning only over this filtered evidence.
The core idea is:

Reduce reasoning complexity by constraining the model to a smaller, evidence-grounded financial context.

This approach does not rely on:

- fine-tuning,
- larger models,
- external APIs,
- or prompt engineering tricks.

Instead, it changes the *information structure* presented to the model.

10.2 Motivation from Layer 2 Findings

The Layer 2 experiments strongly motivated this extension.

The error taxonomy showed that the most common failures were:

- retrieval errors,
- operation-order failures,
- semantic grounding mistakes,
- and contextual confusion.

The context sensitivity experiments also revealed that:

- removing tables severely degraded performance,
- text-only reasoning collapsed almost completely,
- and structured numerical grounding was essential for stable reasoning.

Program-vs-answer evaluation further showed that:

- correct answers did not necessarily imply correct reasoning,
- and symbolic reasoning consistency remained weak even when execution accuracy was correct.

Together, these findings suggested that:

the model frequently fails before arithmetic reasoning even begins.

Thus, the extension specifically targets:

- retrieval ambiguity,
- evidence misalignment,
- and numerical grounding instability.

10.3 Core Insight Behind the Method

Financial documents contain:

- dense numerical information,
- repeated financial metrics,
- multiple fiscal years,
- and semantically similar quantities.

For small models, this creates a major retrieval problem.

For example:

- total assets vs current assets,
- operating income vs net income,
- 2014 revenue vs 2015 revenue,
- percentage vs decimal representation,

may all appear close together inside the same context.

The full-context setup forces the model to:

1. retrieve relevant values,
2. align them semantically,
3. maintain them in memory,
4. and execute arithmetic correctly.

The evidence-grounded approach reduces this burden by removing irrelevant numerical clutter before reasoning begins.

10.4 Method

The implementation uses the `model_input` evidence field already available in the FinQA dataset.

Instead of using:

- full tables,
- pre-text,
- and post-text,

the pipeline extracts only:

- retrieved evidence rows,
- relevant supporting text,
- and question-specific financial statements.

The reasoning prompt itself remains unchanged.

Thus, the only modification is:

- evidence filtering,
- and context reduction.

10.5 Baseline Full-Context Pipeline

The original system used:

- complete financial tables,
- all surrounding financial text,
- and the full document context.

This frequently caused:

- retrieval confusion,
- nearby-number selection errors,
- wrong fiscal-year selection,
- and reasoning drift.

Common failure patterns included:

- mixing totals with subcategories,
- confusing related financial metrics,
- or selecting unrelated nearby quantities.

10.6 Evidence-Grounded Pipeline

The modified system instead supplied:

- only retrieved evidence,
- only question-relevant financial statements,
- and a significantly reduced numerical search space.

Example

Instead of showing:

- an entire multi-row financial table,

the model might receive only:

2014 net revenue = 5735 2015 net revenue = 5829
--

This constrains the reasoning space and reduces distractor interference.

10.7 Why This Helps Small Models

Small models have limited:

- attention capacity,
- retrieval robustness,
- and symbolic grounding ability.

When many similar financial quantities are present simultaneously, the model often:

- retrieves incorrect operands,
- mixes years,
- or loses alignment between question intent and evidence.

The evidence-grounded approach reduces:

- context entropy,
- operand competition,
- and retrieval instability.

This allows the model to focus more capacity on:

- arithmetic execution,
- and reasoning structure.

10.8 Experimental Examples

10.8.1 Example 1 – Retrieval Improvement

Question

“what is the net change in net revenue during 2015 for entergy corporation?”

Evidence-Grounded Context

2014 net revenue = 5735
2015 net revenue = 5829

Model Reasoning

5829 - 5735 = 94

Predicted Answer

94

Gold Answer

94.0

Analysis Restricting the context reduced distractor interference and allowed correct operand retrieval.

10.8.2 Example 2 – Remaining Reasoning Weakness

Question

“what percentage of total facilities as measured in square feet are leased?”

Correct Reasoning

$$\frac{8.1}{56.0}$$

Observed Model Reasoning

$$\frac{2.1}{8.1}$$

Analysis Although the evidence-grounded setup reduced retrieval noise, the model still selected:

- the wrong denominator,
- and incorrect facility quantities.

This shows that:

- evidence grounding reduced retrieval instability,
- but symbolic financial reasoning weaknesses still remained.

10.9 Observed Improvements

The evidence-grounded pipeline produced noticeably better reasoning behavior.

Compared to the original full-context setup:

- operand selection became more stable,
- retrieval drift decreased,
- irrelevant financial quantities were selected less frequently,
- and reasoning traces became more coherent.

Most importantly, the model more consistently selected:

- correct rows,
- correct years,
- and correct financial metrics.

10.10 Important Experimental Insight

The experiments revealed an important distinction between:

- retrieval failures,
- and reasoning failures.

Before Evidence Grounding

Failures were dominated by:

- retrieval instability,
- distractor-number confusion,
- and contextual drift.

After Evidence Grounding

Failures shifted toward:

- operation errors,
- denominator selection mistakes,
- representation mismatches,
- and financial semantic misunderstanding.

This is a meaningful improvement.

It suggests that:

- evidence grounding successfully reduced retrieval-related failures,
- while exposing the remaining weaknesses of the reasoning process itself.

10.11 Financial Importance

In financial AI systems, incorrect evidence retrieval can be extremely dangerous.

Using:

- the wrong fiscal year,
- incorrect liabilities,
- or unrelated financial metrics,

can lead to:

- incorrect valuation,
- misleading trend analysis,
- or flawed investment conclusions.

The proposed evidence-grounded approach improves:

- traceability,
- evidence alignment,
- and interpretability.

Even when the final answer remains incorrect, the reasoning becomes:

- easier to audit,
- easier to debug,
- and more financially trustworthy.

10.12 Limitations

The proposed method still has several limitations:

- it does not guarantee correct symbolic reasoning,
- denominator selection errors still occur,
- arithmetic execution instability remains,
- and evidence retrieval itself is assumed to be correct.

Additionally:

- numerical grounding was verified primarily through number overlap,
- rather than deep semantic financial verification.

Thus, the system improves reasoning reliability, but does not fully solve financial symbolic reasoning.

10.13 What I Would Do Next with More Compute

If additional GPU resources and time were available, the next step would be:

- building a lightweight financial evidence ranker,
- learning question-to-table alignment,
- and adding symbolic program verification before accepting answers.

A promising direction would be:

- extracting intermediate reasoning programs,
- executing them symbolically,
- and rejecting inconsistent reasoning chains automatically.

This could substantially improve:

- interpretability,
- numerical consistency,
- and financial reliability.

10.14 Key Conclusion

This extension suggests that improving financial reasoning for small language models may depend more on:

- evidence grounding,
- retrieval alignment,
- and context structuring,

than on simply increasing model size.

The experiments indicate that:

- many reasoning failures originate before arithmetic begins,

- during the evidence selection stage itself.

Thus, reducing retrieval ambiguity can substantially improve reasoning quality even without:

- fine-tuning,
- larger models,
- or additional compute.

10.15 Overall Conclusion

The Evidence-Grounded Retrieval approach demonstrates that:

Small financial language models fail less because they cannot compute, and more because they struggle to retrieve and align the correct financial evidence before computation begins.

By restructuring the reasoning context around relevant evidence only, the system reduces retrieval instability and improves the reliability of financial reasoning under strict computational constraints.

References

1. FinQA Dataset: <https://github.com/czyssrs/FinQA>
2. Qwen2.5 Model: <https://huggingface.co/Qwen>
3. FinQANet Paper: <https://arxiv.org/abs/2109.00122>
4. Hierarchical Retrieval with Evidence Curation for Open-Domain Financial Question Answering: <https://arxiv.org/abs/2505.20368>
5. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks: <https://arxiv.org/abs/2005.11401>